unfuckdoc — Model Inference Flow

When and where each AI model runs. Everything else is deterministic code.

EMBEDDER vector model (nomic / MiniLM) **LLM** qwen2.5 (generation / judgment) **CODE** deterministic, no model

1 · The guiding principle

Deterministic-first; models only on the ambiguous residual. The pipeline does the exact, cheap, reproducible work (classify, merge, filter, count, aggregate). Models are used only where the task is fundamentally about *meaning* (embeddings) or *language / judgment* (the LLM), and they run as gated, cached steps — never per-row at query time.

2 · The two model types

Model	Job	Produces	Runs
Embedder — nomic-embed-text (default) / all-MiniLM (fallback)	turn text into a vector of meaning	a float vector (768 / 384-d)	ingest + per query
LLM — qwen2.5:7b	read language, output structured JSON / judgment	JSON (attributes, query plan, classification)	ingest (extraction) + per question (planning)

3 · INGEST — runs once, ahead of any query

CODE parse file · classify column types · **canonicalize field names** (alias dictionary)

EMBEDDER **semantic field-name match** — *only* for names the dictionary can't place (gated; 0 on recognized data)

CODE merge / dedup entities by key · consolidate scalar vs array · enrichment joins

EMBEDDER **vectorize free-text fields** — one vector per entity, stored for semantic search (once)

LLM **attribute extraction** — read each description → typed fields (has_garden, property_type). Gated + cached per (text + schema)

Cost is paid once here. Queries are then cheap and exact.

4 · QUERY — runs per request

LLM **NL → query plan** *(planned)* — turn the plain-text question into filters / ranges / geo / tags / a semantic clause. One call per question.

EMBEDDER **RAG grounding** *(planned)* — retrieve the real fields + enum values relevant to the question, so the LLM can't hallucinate.

EMBEDDER **embed the query** — one small vector, if the plan includes a semantic clause.

CODE **semantic ranking** — cosine between the query vector and stored entity vectors (pure math, no model).

CODE **exact execution** — filters, ranges, geo (bbox / polygon), tags, enumeration, joins. Deterministic + auditable.

CODE return the dataset + cosine relevance scores.

5 · Master reference — task → model → when

Task	Model	Phase	Gated?
Column type classification	CODE (statistical)	ingest	—
Field-name canonicalization (dictionary)	CODE (alias table)	ingest	—
Field-name semantic match (residual)	**EMBEDDER**	ingest	yes — only unrecognized names
Merge / dedup / consolidation / enrichment	CODE	ingest	—
Vectorize free text	**EMBEDDER**	ingest	once per entity, cached
Attribute extraction (has_garden, risk, …)	**LLM**	ingest	yes — per entity, cached
Embed the query	**EMBEDDER**	query	once per search
Semantic ranking (cosine)	CODE	query	—
NL → structured query plan *(planned)*	**LLM**	query	yes — once per question
RAG grounding (retrieve schema/values) *(planned)*	**EMBEDDER**	query	once per question
Filters · ranges · geo · tags · enumeration · aggregation	CODE	query	—

> **Numbers by code, judgment by the LLM.** For aggregate classification (e.g. "high/low loan risk" from many financial records): compute the features deterministically (income, debt-to-income, missed payments) with CODE, then hand the *summary* to the LLM for the judgment — never dump raw rows into the model. For regulated decisions, keep an auditable scored model as the decider and use the LLM to summarize / explain / flag.

6 · Deployment & swapping models

Both models sit behind one interface each and are chosen by environment variable — any OpenAI-compatible endpoint works (local Ollama, OVH AI Endpoints, vLLM):

Env	Effect
`EMBED_BASE_URL` / `EMBED_MODEL`	use a remote embedder (else in-process MiniLM)
`LLM_BASE_URL` / `LLM_MODEL`	enable the LLM (else attribute extraction is off)
`UNFUCK_NO_EMBED=1`	disable embeddings entirely (deterministic-only)

Local dev today: nomic-embed-text + qwen2.5:7b on Ollama (`http://localhost:11434/v1`), Metal-accelerated. Prod option: OVH AI Endpoints, EU-resident.
